



Main Menu

MainMenu.

- Первая сцена, которая загружается сразу после экрана загрузки - Главное меню.

Снизу текст-примечание. Справа текущие рекорды игрока, которые сохраняются в Json файл и подгружаются из SaveManager'a при каждом включении данной сцены. Слева классическая панель: начать игру, настройки, выход.

Начать игру - вызывает `GameFlowManager.Instance.StartRun()`, который сам выбирает случайную карту через `LevelManager`.

Настройки - вызывают окошко настроек. Более подробно на странице `Settings System`.

Выход - закрывает данный application.

- На фоне пока что находиться серый экран. Как только игра будет достаточно функционировать - я сыграю один забег, запишу на видео и поставлю данный ролик на фон, затемнив картинку. Возможно сделаю небольшой паралакс эффект от движения мышью.



Главное меню (незаконченное).



SettingsController.

Buttons.

```

1 using UnityEngine;
2
3 public class StartGame : MonoBehaviour
4 {
5     public void StartGameClick()
6     {
7         if (SettingsMenuController.Instance.gameObject.activeInHierarchy)
8         {
9             SettingsMenuController.Instance.Hide();
10        }
11
12        GameFlowManager.Instance.StartRun();
13    }
14 }

```

```

1 using UnityEngine;
2 using UnityEngine;
3
4 public class Settings : MonoBehaviour
5 {
6     public void SettingsClick() =>
7         SettingsMenuController.Instance.Show();
8 }

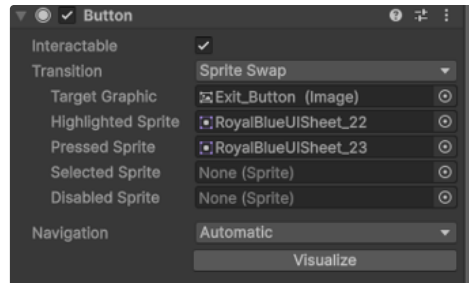
```

```

1 using UnityEngine;
2
3 public class ExitGame : MonoBehaviour
4 {
5     public void ExitButton()
6     {
7         FadeManager.FadeOutThen(() => {
8             #if UNITY_EDITOR
9                 UnityEditor.EditorApplication.isPlaying = false;
10            #else
11                Application.Quit();
12            #endif
13        }, 1.5f);
14 }

```

У меня есть спрайты для того, чтобы при наведении на кнопку и нажатии, спрайт кнопки менялся. Однако этого мне показалось мало и я решил сделать кнопки более живыми и интересными.



В попытках что-то придумать, у меня получился эффект прилипания кнопок к курсору мыши.

Запоминаем `RectTransform` и текущую позиции кнопки. Подключаем `IPointerEnterHandler`, `ExitHandler` и будем просто менять флаг, когда курсор будет наводится на объект, или уходить за границы объекта.

`Vector2 mousePos` - экранные координаты мыши в пикселях (например: 0 - 1920). Позиция кнопки на `Canvas` - локальны координаты внутри UI. Сравнивать их напрямую нельзя, кнопка будет вести себя не верно.

Метод `RectTransformUtility.ScreenPointToLocalPointInRectangle` берет экранную позицию мыши в пикселях, учитывает камеру, отслеживающую UI и размеры `Canvas`. После чего выдаёт позицию курсора относительно центра самой кнопки. (0, 0) = центр кнопки, (15, 0) = правее центра. Это запишется в `localMousePos` поле.

Теперь делаем ограничение движения кнопки через `Vector2.ClampMagnitude`(локальная позиция мыши, максимальный сдвиг). `Clamp` берёт вектор и проверяет его длину. Если мышка ушла далеко (условно на 100 пикселей), а `moveAmount` равен меньше, то клэмп пропорционально сожмёт вектор до `moveAmount`. Как итог, мы получим вектор, куда надо двигаться, но не более того, что указан.

Теперь получим таргет - возьмём стандартную позицию кнопки, прибавим вектор3 (максимальный вектор.X, максимальный вектор.Y, 0). Например, позиция кнопки -500, 200. Если не прибавлять `originalPosition`, то мы получим... Например → (10, -5). В таком случае кнопка просто улетит на эти координаты. Поэтому в итоге мы получим (-490, 195), как координаты, куда кнопка должна подвинуться.

Зачем конвертировать в Vector3? → rectTransform.localPosition мы будем двигать через SmoothDamp, который имеет Vector3.

В отличие от Mathf.Lerp, SmoothDamp симулирует реальную физическую пружину или амортизатор, а не линейный или процентный шаг. Присваиваем позиции нашей кнопки SmoothDamp(текущая позиция, координаты куда двигаться, velocity - информацию о скорости движения в предыдущем кадре (которая нужна для соблюдения правил энергии), время сглаживания (приблизительное время, за которое кнопка пройдёт большую часть пути))

Ну а если курсор мыши находится вне границ кнопки - мы аналогично сбрасываем позицию.

```
1 using UnityEngine;
2 using UnityEngine.EventSystems;
3 #if ENABLE_INPUT_SYSTEM
4 using UnityEngine.InputSystem;
5 #endif
6
7 public class ButtonFollowMouse : MonoBehaviour, IPointerEnterHandler, I
8 {
9     [Header("Settings")]
10    [SerializeField] public float moveAmount = 10f;
11    [SerializeField] public float smoothTime = 0.1f;
12    [SerializeField] public Camera camera;
13
14    private RectTransform rectTransform;
15    private Vector3 originalPosition;
16    private Vector3 velocity;
17
18    private bool isHovered = false;
19
20    private void Awake()
21    {
22        rectTransform = GetComponent<RectTransform>();
23        originalPosition = transform.localPosition;
24    }
25    void Update()
26    {
27        if (isHovered)
28        {
29            Vector2 mousePos;
30            #if ENABLE_INPUT_SYSTEM
31                mousePos = Mouse.current.position.ReadValue();
32            #else
33                mousePos = Input.mousePosition;
34            #endif
35
36            Vector2 localMousePos;
37            RectTransformUtility.ScreenPointToLocalPointInRectangle(
38                rectTransform,
39                mousePos,
40                camera,
41                out localMousePos
42            );
43
44            Vector2 clamped = Vector2.ClampMagnitude(localMousePos, mov
45            Vector3 targetPos = originalPosition + new Vector3(clamped.
```

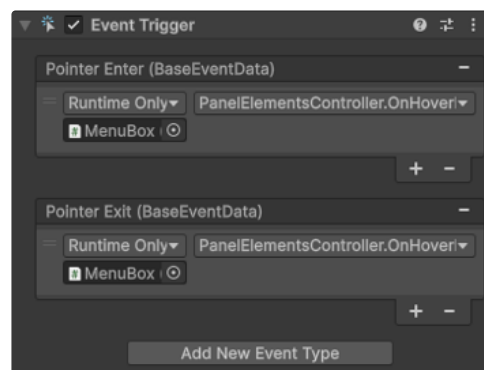
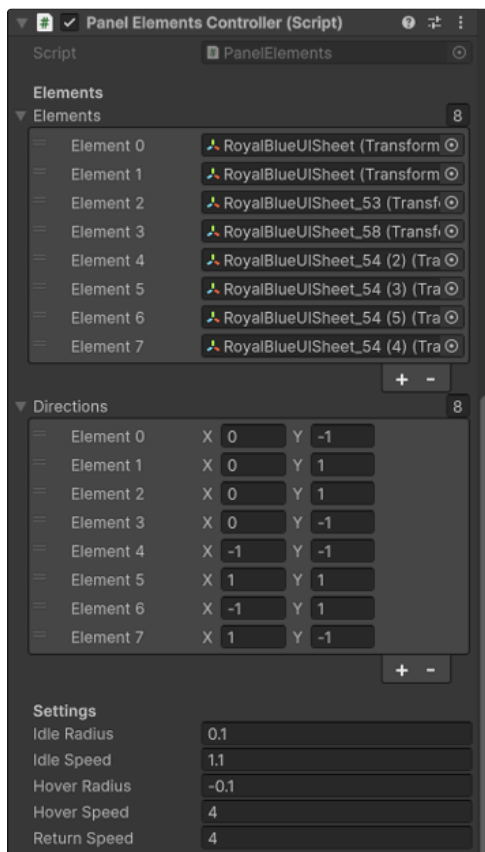
```

47         rectTransform.localPosition = Vector3.SmoothDamp(rectTransf
48             targetPos, ref velocity, smoothTime);
49     }
50     else
51     {
52         rectTransform.localPosition = Vector3.SmoothDamp(rectTransf
53             originalPosition, ref velocity, smoothTime);
54     }
55 }
56
57 public void OnPointerEnter(PointerEventData eventData)
58 {
59     isHovered = true;
60 }
61
62 public void OnPointerExit(PointerEventData eventData)
63 {
64     isHovered = false;
65 }
66 }

```

Animations Fragments.

- i** Чтобы было ещё более “живо”, было решено сделать анимацию остальных графических деталей. Идея в том, что все они двигаются в противоположном от меню направлении, а потом обратно. В случае наведения на блок меню - они прижимаются к нему и застывают.



Здесь isHover меняется иначе. Я для эксперимента повесил EventTriger, у которого встроены PointerEnter, PointerExit. И просто передал им ссылку на методы изменения isHover.

Придётся немного поработать ручками. Поэтому закидываем все элементы в нужном порядке, а потом для каждого элемента указываем его направление.

В массиве дирекшинс мы будем хранить направления: -1, 0, 1 для X, Y. Это будет вектор направления, в сторону которого могут двигаться элементы.

В Start пакуем стартовые позиции в массив, указываем прогресс как 0 (показывает насколько близко элемент к разрешенному радиусу idleRaduis), directionSign - знак направления (-1, 1). В старте мы изначально всем раздаём 1, запуская маятник вперёд (пульсацию наружу).

В Update проверяем, если курсор наведён, то устанавливаем таргет позицию для каждого элемента. В данном случае таргет позиция вычисляется так: стартовая позиция + Вектор3(направлениеX, направлениеY, 0) умноженные на радиус (в нашем случае он минусовой, чтобы элементы сильнее прижались к панели меню (внутрь)).

К примеру: верхний правый объект (синий угловой узор).

Его направление = (1, 1)
Получается правая верхняя плоскость. Настоящая стартовая позиция: (-3.1, 3.5). Считаем таргет точку:
 $-3.1, 3.5 + 1, 1, 0 * -0.1f = (-3, 3.4)$.
Это точка в пространстве, что находится ближе к блоку меню, чем стартовая позиция объекта.



Далее сдвигаем через
Vector3.Lerp(текущая позиция, куда сдвигаться, дельтаТайм * скорость) → линейная интерполяция. Стартовое движение объекта будет

достаточно быстрым, а вот по приближению к финальной точке - элемент будет замедляться и плавно встанет на место.

```
1 using UnityEngine;
2
3 public class PanelElementsController : MonoBehaviour
4 {
5     [Header("Elements")]
6     public Transform[] elements;
7     public Vector2[] directions;
8
9     [Header("Settings")]
10    public float idleRadius = 0.2f;
11    public float idleSpeed = 1.15f;
12    public float hoverRadius = -0.1f;
13    public float hoverSpeed = 4f;
14    public float returnSpeed = 4f;
15
16    private Vector3[] startPositions;
17    private bool isHover = false;
18    private float[] progress;
19    private int[] directionSign;
20
21    private Vector3[] returnPositions;
22
23    void Start()
24    {
25        if (elements == null || directions == null || elements.Length !=
26            directions.Length)
27        {
28            Debug.LogError($"[PanelElementsController] Ошибка настройки на
29                стroyки. Количество элементов ({elements?.Length}) не с
30                оответствует количеству направлений ({directions?.Length}).
31                Анимация полностью отключена.");
32
33            enabled = false;
34            return;
35        }
36
37        startPositions = new Vector3[elements.Length];
38        progress = new float[elements.Length];
39        directionSign = new int[elements.Length];
40        returnPositions = new Vector3[elements.Length];
41
42        for (int i = 0; i < elements.Length; i++)
43        {
44            startPositions[i] = elements[i].localPosition;
45            progress[i] = 0f;
46            directionSign[i] = 1;
47
48            if (directions.Length <= i)
49            {
50                directions = new Vector2[elements.Length];
51                directions[i] = directions[i].normalized;
52            }
53        }
54
55        void Update()
56        {
57            for (int i = 0; i < elements.Length; i++)
58            {
59                Vector3 targetPos;
60
61                if (isHover)
```

```

59         {
60             targetPos = startPositions[i] + new Vector3(directions[
61                 elements[i].localPosition = Vector3.Lerp(elements[i].lo
62             }

```

Если же наведения мыши нету, то у элементов происходит плавная пульсирующая анимация.

Как только мышь уведется - наша задача плавно вернуть элементы на их места. В это время элемент находится очень далеко от своей траектории покачивания. В случае, если мы не укажем плавное возвращение, то элемент будет телепортироваться на нужную позицию.

Сначала сделаем проверку: Если (локальная позиция - (стартовая позиция + вектор направлений * радиус idle состояния * прогресс))) переведенная в магнитуду больше 0.01f) ...

Операция рассчитывает точку, где элемент должен был бы сейчас находиться в цикле покачивания.

Вычитает из текущей позиции кнопку эту целевую точку (А-Б).

А свойство .magnitude считает физическое расстояние в пикселях\единицах между этими двумя точками (длину вектора). Если расстояние больше 0.01f → значит элемент сбит с траектории покачивания.

Теперь плавно возвращаем его на нужную точку. Вычисляем таргет → стартовая позиция + вектор направления * радиус покачивания * прогресс.

После чего будем двигать через Lerp(позиция, таргет, дельтаТайм * скорость возвращения).

Как только элемент доплывает обратно и расстояние становится меньше 0.01f - управление переходит в ветку else, маятник покачивания подхватывает элемент и он снова плавно движется туда-сюда.

На старте прогресс = 0, дирекшинСайн = 1 (наружу). Перевычисляем прогресс: $0 += 1 * 1.15 * 0.016$ (в идеальных условиях). = 0,0184. Каждую секунду прогресс будет расти.

Проверяем, не стал ли прогресс равен 1ке - не достиг ли он конца разрешенной зоны. Если достиг → устанавливаем прогресс в 1, а дирекшинСайн (направление) устанавливаем на -1 (внутрь).

Если прогресс достиг нуля (достиг границ внутреннего края) - то устанавливаем его в 0, а направление устанавливаем наружу.

Таргет = стартовая + вектор направления * доступный радиус * прогресс (направление (внутри, наружу), скорость, время).

Например: $(-3.1, 3.5) + 1.1 * 0.2 * 0.018 = (-3.10036, 3.50036)$.

Просто устанавливаем таргет как текущую позицию. Почему не плавный Lerp? - Разрыв между текущим положением элемента и вычисленным таргетом составляет менее 0.01 пикселя. Что крайне мало. Lerp бы наоборот сломал логику. Разница = 0.0036. Благодаря огромной скорости Update и низкой разнице позиций, перемещение всё равно выглядит очень плавным.

Забавно, что логика состоит во многом из направлений и просто смены плюса на минус у числа. Ведь если бы у нас были -1, -1 (движение вниз влево), то для данного элемента бы получили например -3.0964 по X, начав движение в обратную сторону. К слову, для противоположного (нижнего-левого) элемента -1, -1 будет двигать на старте его наоборот от панели. А потом направление сменится на 1, 1. Простая, но забавная логика 😊

```
1         else
2         {
3             if ((elements[i].localPosition - (startPositions[i] + n
4             {
5                 targetPos = startPositions[i] + new Vector3(directionSign[i] * radius * progress);
6                 elements[i].localPosition = Vector3.Lerp(elements[i].localPosition, targetPos, Time.deltaTime);
7             }
8             else
9             {
10                progress[i] += directionSign[i] * idleSpeed * Time.deltaTime;
11
12                if (progress[i] >= 1f)
13                {
14                    progress[i] = 1f;
15                    directionSign[i] = -1;
16                }
17                else if (progress[i] <= 0f)
18                {
19                    progress[i] = 0f;
20                    directionSign[i] = 1;
21                }
22
23                targetPos = startPositions[i] + new Vector3(directionSign[i] * radius * progress);
24                elements[i].localPosition = targetPos;
25            }
26        }
27    }
28
29
30    public void OnHoverEnter()
31    {
32        isHover = true;
33    }
34
35    public void OnHoverExit()
36    {
37    }
```

```
37     isHover = false;
38 }
39 }
```

⚠ Блок ниже также присутствует на странице Save System.

Records Panel и анимация бегущих чисел.

❗ В скрипте панели рекордов, при включении объекта, мы получаем данные из менеджера и закидываем их на интерфейс.

```
1 using TMPro;
2 using UnityEngine;
3 using UnityEngine.UI;
4 public class RecordsPanel : MonoBehaviour
5 {
6     [SerializeField] public TextMeshProUGUI bestScoreText;
7     [SerializeField] public TextMeshProUGUI mostWavesText;
8     [SerializeField] public TextMeshProUGUI totalWavesText;
9     [SerializeField] public TextMeshProUGUI totalKilledText;
10    private void OnEnable()
11    {
12        var stats = SaveManager.Instance.Data.stats;
13        bestScoreText.text = stats.bestScore.ToString();
14        mostWavesText.text = stats.mostWaves.ToString();
15        totalWavesText.text = stats.totalWaves.ToString();
16        totalKilledText.text = stats.totalKilled.ToString();
17    }
18 }
```

❗ Статичный текст мне показался скучным и я думал что бы добавить... Мне пришло на ум сделать простенькую анимацию с переборкой цифр. Каждые пару секунд идёт переборка цифр в случайном порядке, а потом останавливается на нужном значении.

У меня будет бесконечный цикл, который будет крутить короутину, а после запускать задержку перед повторной анимацией.

Сначала мы собираем все данные себе в массив. Далее попробуем из текста получить число. К слову, текст "MaxWave" - у меня идёт отдельным объектом, и Regex можно было не использовать, но пускай будет. Regex.Match - пробегается по тексту с определенным шаблоном, в данном случае - @"d+"

@ - технический символ, чтобы работал обратный слеш

d - любая цифра (0-9)

+ - одна или более подряд

То есть в определенный момент, если бы текст был цельным: Максимум волн: 237, то

метод пробежался бы по массиву символов, дошёл до 2, и потом собирал бы цифры, пока снова не начались бы другие символы. В итоге мы получим 237. У меня цифры идут отдельным текстом, но как защиту - предусмотреть можно. Далее мы парсим строку в инт тип.

Запускаем сам цикл, длинной в некоторое время. Через for присваиваем в каждое поле случайное число. Но не абы какое, а равное по ширине тому, что изначально у нас находится в поле. Если волн 237 - значит анимация будет от 100 - 999. После чего мы просто выводим число на экран. И ожидаем некоторую задержку. Она кажется совсем маленькой = 0.05 секунды, однако без неё - анимации бы не было вообще. While бы просто срабатывал ультра быстро и выводил бы мгновенно прошлый результат, а так мы наблюдаем анимацию бегущих цифр.

В конце просто возвращаем на экран реальные значения.

```
1 using System.Collections;
2 using TMPro;
3 using UnityEngine;
4 using System.Text.RegularExpressions;
5 using System.Collections;
6 public class RecordsAnimator : MonoBehaviour
7 {
8     [Header("Fields")]
9     public TMP_Text[] recordFields;
10    [Header("Settings")]
11    public float initialDelay = 2.5f;
12    public float animationDuration = 2f;
13    public float repeatDelay = 7f;
14    public float updateInterval = 0.05f;
15    void Start()
16    {
17        StartCoroutine(AnimateRecordsLoop());
18    }
19    IEnumerator AnimateRecordsLoop()
20    {
21        yield return new WaitForSeconds(initialDelay);
22        while (true)
23        {
24            yield return StartCoroutine(AnimateRecords());
25            yield return new WaitForSeconds(repeatDelay);
26        }
27    }
28    IEnumerator AnimateRecords()
29    {
30        float timer = 0f;
31        int[] targetNumbers = new int[recordFields.Length];
32        for (int i = 0; i < recordFields.Length; i++)
33        {
34            string text = recordFields[i].text;
35            int num = 0;
36            Match match = Regex.Match(text, @"\d+");
37            if (match.Success)
38            {
39                num = int.Parse(match.Value);
40            }
41            targetNumbers[i] = num;
42        }
43    }
44 }
```

```
43     while (timer < animationDuration)
44     {
45         timer += updateInterval;
46         for (int i = 0; i < recordFields.Length; i++)
47         {
48             int digits = targetNumbers[i].ToString().Length;
49             int randomValue;
50             switch (digits)
51             {
52                 case 1: randomValue = Random.Range(0, 10); break;
53                 case 2: randomValue = Random.Range(10, 100); break;
54                 case 3: randomValue = Random.Range(100, 1000); break;
55                 case 4: randomValue = Random.Range(1000, 10000); break;
56                 case 5: randomValue = Random.Range(10000, 100000); break;
57                 default: randomValue = targetNumbers[i]; break;
58             }
59             recordFields[i].text = randomValue.ToString();
60         }
61         yield return new WaitForSeconds(updateInterval);
62     }
63     for (int i = 0; i < recordFields.Length; i++)
64     {
65         recordFields[i].text = targetNumbers[i].ToString();
66     }
67 }
68 }
```